

Assignment 3

May 18, 2026

IMAGES. All images may be found in the DATASETS folder https://drive.google.com/drive/folders/1fUo6TQmvkCPWg9MVaEQTyx3tOZ0yTEkG?usp=share_link.

Independent Reading. Read Chapters 11 and 12 from Szeliski's book.

Independent Reading. NeRF: <https://arxiv.org/pdf/2003.08934> and

3DGS: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

1. **(3 pts) Homography Estimation.** You will be using the Checkerboard Images for this problem. Take a pair of images and estimate the Homography matrix H between them. You can use the `findChessboardCorners` function in OpenCV to extract the corner points. Make sure there is a one-to-one correspondence between the corner points in the two images. Report the homography matrix for the image pairs 2 and 9. Specifically, you need to estimate H , where $P_9 = HP_2$, where P_2 and P_9 are the points extracted in the images 2 and 9, respectively. **You should not use built-in functions to estimate the homography matrix.**
2. **(3 pts) Fundamental Matrix Estimation.** In this problem, you will be using the stereo image pairs `viprectification_deskLeft.png` and `viprectification_deskRight.png`, provided in the DATASETS folder. Load the images and detect the key points, i.e. corner points, using any predefined corner detector in Python (refer to Assignment 1). Next, you will obtain the feature descriptors of the corners using the ResNet50 model in Assignment 3. Use 8×8 patches around each of the corner points to extract the features. If required, make sure to pad or resize the patches before inputting them into the model. Using the matched points, employ the 8-point algorithm to obtain the fundamental matrix. Use random subsets of 8 points to estimate a set of fundamental matrices. Then, for each fundamental matrix, compute the mean error on the rest of the matched points using the Sampson distance. Report the fundamental matrix that produced the least error and also report the mean error. **You should not use built-in functions to estimate the fundamental matrix.** Sampson distance is defined as: $\frac{(x^T F x')^2}{\|F^T x\|_2^2 + \|F x'\|_2^2}$, where x and x' are the 2D points on the image plane in the image pair.

3. **(4 pts) 3D Gaussian Splatting.** Capture a short video (30 s to 1 min) in and around the UCR campus using your phone, favoring a mostly static scene with sufficient parallax (*i.e.*, walk around the subject rather than panning in place). Extract frames at a sensible rate (roughly 300 frames) and run COLMAP (or an equivalent SfM pipeline) to recover per-frame camera poses $\{\mathbf{R}_i, \mathbf{t}_i\}$, intrinsics $\mathbf{K}$. These poses serve as the *ground-truth (GT) trajectory*. Optimize a 3D Gaussian Splatting model on the captured frames using following 3DGS, then render an image at *every* GT pose and assemble the renders, in capture order, into a video at the same frame rate as your input sequence. Finally, produce a single side-by-side video showing the captured GT frames on the left and the 3DGS-rendered frames on the right, time-aligned frame for frame. Report the number of training iterations, the final number of Gaussians, PSNR/SSIM on training views, and briefly discuss the failure modes you observe. *Heads up: you might have to create a singularity for this, naive conda install may not work.* **Please submit the video or include a link to the same in your report. Please use the compute resources allocated to you for your projects for this question.**

Submission Protocol. For all problems, you must submit a Jupyter notebook, *i.e.*, `.ipynb` file. You are recommended to use Colab. You should add comments to your codes to make them reader-friendly. All codes must be uploaded to separate folders named after the problem number. Keep all the images necessary to run a code in the same folder as the code while you are submitting. The zip file to be uploaded must have your name.